Critic-Matched Movie Prediction

Analytic, boosted-tree, and neural designs, each with a raw and a z-score track

Portfolio project — technical report

Rotten Tomatoes critic reviews and Letterboxd member ratings (Kaggle)

Abstract

Two fully parallel projects predict a person's rating of a film from other raters: critic pseudo-users on Rotten Tomatoes (301 test critics, 43,344 paired episodes) and real members on Letterboxd (300 test members, 41,166 paired episodes). Three designs compete on identical held-out episodes in each project: an analytic movie-mean-centered formula (with a second variant restricted to the 10 most-aligned-or-anti-aligned peers), an XGBoost regressor, and a residual neural-network ensemble – all sharing one engineered feature contract that now includes, beyond similarity and consensus, a real movie-facet contract (genre, theme, studio, director, actor, decade, language, country) joined from the gsimonx37/letterboxd Kaggle metadata dump, with a per-user affinity term for each facet. On Rotten Tomatoes, full-history RMSE is 0.796 (Design 1), 0.770 (XGBoost), 0.776 (neural net), all beating the 0.826–0.830 flat baselines. On Letterboxd, full-history RMSE is 1.507, 1.455, 1.468 against a 1.639 flat baseline. Every design also trains a parallel z-score track (each rater standardized to their own scale, predicting pure deviation, converted back before scoring) to test whether the gain is calibration or taste-matching; the result is not uniform across datasets or designs, reported as found (§6).

1 Data

1.1 Rotten Tomatoes

The source is `andrezaza/clapper-massive-rotten-tomatoes-movies-and-reviews`. Fractions, letter grades, percentages, and star ratings are parsed and quantized to $\{0, 1, 2, 3, 4, 5\}$, the *standardized raw score* – the prediction target. Critics with at most five parseable scores are excluded, name variants are deduplicated, and repeat critic–movie cells are averaged.

Data funnel	**Count**
Raw review rows (start)	1,444,963
Distinct critics with a parseable score	8,623
After the low-volume critic floor (> 5 scored reviews)	4,393 critics, 1,262,747 rows
Parsed, standardized scored reviews used for modelling	992,954
Distinct movies with any scored review	58,736
Neighbour pool / pseudo-users (≥ 10 distinct movies)	2,894 critics
Movies in the final app catalog (most-reviewed)	1,000
Critics appearing in that catalog	3,387

Table 1: From 1.4M raw rows to the modelling pool and the app catalog.

The data retain a diagnostic z-score:

$$z_{c,i} = \frac{r_{c,i} - \mu_c}{\sigma_c}. \tag{1}$$

Here $r_{c,i}$ is critic c's standardized raw score for movie i; μ_c is critic c's all-time mean score; and σ_c is critic c's all-time standard deviation. The *Tomatometer* is the percentage of reviews labelled Fresh, not a mean 0–5 rating; baseline B2 fits $t_m = aT_m + b$ (a linear calibration of movie m's Tomatometer percentage onto the raw score scale).

Movie facets. RT's own `genre` column is a comma-separated multi-genre string, parsed in full (not just the first entry). This is joined to the gsimonx37/letterboxd Kaggle metadata dump (`genres`, `themes`, `studios`, `actors`, `crew`, `countries`, `languages` – posters excluded, unneeded) by normalized (title, year), with title-only fallback when a year is missing (RT's own release-date field is populated for only 30,512 of 142,052 movies). The join matches **69.2%** of RT's full movie catalog and **99.3%** of the 1,000-film app catalog (popular films skew toward better metadata coverage); unmatched films fall back to RT's own genre string and empty facets elsewhere, reported honestly rather than assumed complete.

1.2 Letterboxd

To test whether real, dense per-user rating histories beat the sparse critic pseudo-users above, a fully parallel second project was built on Letterboxd community ratings (Kaggle, ˜11.08M ratings), isolated in its own directory tree (`src/letterboxd/`, `data/letterboxd/`, `results/letterboxd/`) sharing no data, models, or code with Rotten Tomatoes. The export yields 7,420 members with at least five rated films across 286,069 films – 11,078,045 ratings total. Ratings are native on a 1–10 scale (no score parsing, no Tomatometer feature).

The same gsimonx37 join is repeated independently (title, year; LB's own `movie_id` slug already encodes the year, so the join is more reliable here) and matches **94.3%** of LB's full catalog and **99.8%** of the 1,000-film app catalog. LB's own `genres` column (a JSON list, already multi-genre) is the fallback when gsimonx37 misses.

2 Equations and concepts

2.1 Similarity and magnitude

Let $O_{u,c}$ be movies rated by pseudo-user/visitor u and peer (critic/member) c among the sampled seen movies. Pearson correlation is

$$\rho_{u,c} = \frac{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})(r_{c,i} - \bar{r}_{c,O})}{\sqrt{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})^2} \sqrt{\sum_{i \in O_{u,c}} (r_{c,i} - \bar{r}_{c,O})^2}}. \tag{2}$$

Small overlaps are shrunk toward zero:

$$s_{u,c} = \rho_{u,c} \frac{\min(n_{u,c}, k)}{k}, \qquad n_{u,c} = |O_{u,c}|,\ k = 8. \tag{3}$$

The rating-magnitude multiplier (a least-squares through-origin mapping from peer to user ratings) is

$$g_{u,c} = \frac{\sum_{i \in O_{u,c}} r_{u,i} r_{c,i}}{\sum_{i \in O_{u,c}} r_{c,i}^2}. \tag{4}$$

Pearson correlation is affine-invariant, so $\rho_{u,c}$ (and $s_{u,c}$) is *identical* whether computed in raw units or z-units; only $g_{u,c}$ and the target consensus differ between the raw and z-score tracks (§2.5).

2.2 Movie-mean-centered prediction and the top-|sim| variant

For target movie m, let C_m be other peers who rated it, with movie mean $\bar{r}_m = \frac{1}{|C_m|}\sum_{c\in C_m} r_{c,m}$. The full-neighbourhood prediction is

$$\hat{r}_{u,m} = \frac{\sum_{c\in C_m}\left(|s_{u,c}|\bar{r}_m + s_{u,c}(r_{c,m} - \bar{r}_m)\right) g_{u,c}}{\sum_{c\in C_m}|s_{u,c}|}, \tag{5}$$

clipped to the dataset's rating range; a zero denominator falls back to $\bar{r}_m$.

A second, new variant restricts C_m to $C_m^{(k)} \subseteq C_m$, the $k = 10$ peers with the largest $|s_{u,c}|$ – both strongly aligned ($s_{u,c} \gg 0$) *and* strongly anti-aligned ($s_{u,c} \ll 0$) peers, not just the most positively-aligned like baseline B4:

$$C_m^{(k)} = \underset{c\in C_m}{\arg\,\text{top-k}}\ |s_{u,c}|, \qquad \hat{r}_{u,m}^{(k)} = \frac{\sum_{c\in C_m^{(k)}}\left(|s_{u,c}|\bar{r}_m^{(k)} + s_{u,c}(r_{c,m} - \bar{r}_m^{(k)})\right) g_{u,c}}{\sum_{c\in C_m^{(k)}}|s_{u,c}|}, \tag{6}$$

where $\bar{r}_m^{(k)}$ is the mean over the restricted set $C_m^{(k)}$. Equation 6 is otherwise identical to Equation 5 – same shrinkage, same magnitude term – just over a smaller, alignment-selected neighbourhood. The original full-neighbourhood formula is unchanged and remains the default; the top-|sim| variant is purely additive, computed identically for both datasets.

2.3 Decile features

For the trained designs, each peer's target-movie deviation is its leave-one-out all-time mean subtracted from its target score. Peers are ranked by $s_{u,c}$ and their $s_{u,c} \times$ deviation products are summarized (mean, count, standard deviation) in ten similarity-ranked deciles – 30 columns, identical construction for both datasets and both raw/z tracks.

2.4 Movie-facet affinity

Beyond similarity, every trained design sees a rich per-facet contract built from the gsimonx37 join (§1.1, §1.2). For each of eight facets $f \in \{\text{genre, decade, theme, language, country, studio, director, actor}\}$, let $F_f(i)$ be movie i's set of values for that facet (e.g. $F_{\text{genre}}(i) = \{\text{Drama, Thriller}\}$). For a seen-set S (with per-film deviations d_i – $d_i = r_{u,i} - \bar{r}_u$ on the raw track, or the seen z-value on the z-track, see §2.5) and target m:

$$\text{dev}_f(u,m) = \begin{cases} \frac{1}{|H_f|}\sum_{i\in H_f} d_i & H_f \neq \varnothing \\ 0 & H_f = \varnothing \end{cases}, \qquad H_f = \{i \in S : F_f(i) \cap F_f(m) \neq \varnothing\}, \tag{7}$$

$$\text{cnt}_f(u,m) = \log(1 + |H_f|). \tag{8}$$

dev_f is the personalization signal: does this user's seen history over- or under-rate films sharing facet f with the target? It is built *only from seen films* ($H_f \subseteq S$), so it is leakage-free by construction – the target's own rating is never consulted. cnt_f is a confidence weight (how many seen films actually informed dev_f). Sixteen columns (one dev/cnt pair per facet) are added for both projects identically.

The two lowest-cardinality facets, genre and decade, additionally get a target-side multi-hot descriptor – a fixed-width one-hot-style vector over a top-K vocabulary (genre: $K = 30$; decade: $K = 20$; the $(K{+}1)^{\text{th}}$ slot is a catch-all "other") – so the trained models can also learn a global base-rate effect ("comedies score higher on average"), not just the personalization term. The remaining six higher-cardinality facets (theme, language, country, studio, director, actor) rely on the affinity pair alone: a one-hot over thousands of distinct studios or actors would be an unmanageable, overfitting-prone width for a catalog this size, while the affinity term already carries the useful personalization signal regardless of cardinality. A small numeric tail – $\log(1 + \text{runtime})$, the gsimonx37 dataset's own average member rating for that film, and $\log(1 + n_{\text{themes/languages/countries}})$ – completes the contract. In total the feature row grows from 38 (RT, with Tomatometer) / 37 (LB) columns to 111 / 110.

2.5 The z-score track

Every design also trains a parallel z-score variant to test directly whether its gain over the baselines is level-calibration or genuine taste-matching. Each rater is standardized to their own scale using **two different conventions**, deliberately:

- **The user** (the fake pseudo-user offline, or the real visitor in the app) is standardized by the mean/std of *only the ratings sampled as seen for this episode* – never a critic/member's all-time stats – since a real visitor only ever has their own seen films to standardize against: $z_{u,i} = (r_{u,i} - \mu_u)/\sigma_u$ where μ_u, σ_u come from that episode's seen set.
- **Peers** (critics/members) are standardized by their *all-time* mean/std – Equation 1 for RT, the analogous all-time computation for LB.

The model predicts pure deviation $\hat{z}$ in this mixed z-space, then converts back to the raw scale before scoring:

$$\hat{r} = \text{clip}(\mu_u + \sigma_u \hat{z},\ \text{range})\,. \tag{9}$$

user_mean ≈ 0 by construction on this track (the level was removed), and the facet-affinity deviations d_i in Equation 7 use $z_{u,i}$ directly (already $\approx$ 0-mean) instead of $r_{u,i} - \bar{r}_u$. RMSE is always reported after Equation 9, so raw-track and z-track numbers are directly comparable (§6.3). An episode is skipped on the z-track only if its seen ratings have zero variance ($\sigma_u = 0$, can't standardize).

2.6 Metrics

Primary error is

$$\text{RMSE} = \sqrt{\frac{\sum_{j=1}^{N} (\hat{r}_j - y_j)^2}{N}}, \tag{10}$$

where j indexes an episode. Two secondary metrics (§8) check whether the gain is calibration or reordering: within-user Spearman rank correlation, and precision@10 (overlap between a user's predicted and true top-10 held-out films).

3 Design 1: analytic formula

3.1 Raw

Design 1 is Equation 5 (full neighbourhood) and its new top-|sim| variant, Equation 6, computed identically on both datasets from the same similarity/magnitude substrate (§2). No trained parameters beyond the validation-selected shrinkage constant $k = 8$ (Equation 3) and the neighbourhood size $k = 10$ for the top-|sim| variant. B4 (the existing baseline) selects the k most *positively*-aligned peers and averages unweighted; the new Design-1 variant selects by $|s_{u,c}|$ (so it can include strongly anti-aligned peers) and applies the full weighted formula, not a plain average.

3.2 Z-score

Both the full-neighbourhood and top-|sim| formulas are recomputed against the peer pool's all-time z-values (Equation 1) instead of raw scores, with $g_{u,c}$ and the target consensus in z-units; the shrunk similarity $s_{u,c}$ is unchanged (affine invariance, §2). The z-space prediction is converted back via Equation 9 using *this episode's* seen-set μ_u, σ_u before clipping and scoring.

4 Design 2: XGBoost

4.1 Raw

An XGBoost regressor (native Booster API) over the full 111/110-column feature row (§2): the 30 decile columns, seen-count/overlap/dispersion/reviewer-count, mapped Tomatometer (RT only), the 8 facet-affinity dev/cnt pairs, the 51/41-column genre+decade multi-hot, and the numeric tail. `genre_id` (RT's own single-genre embedding index, kept for continuity with Design 3) enters as a plain ordinal column; missing Tomatometer values are handled natively by XGBoost's missing-value split direction. Trained with early stopping on validation RMSE.

4.2 Z-score

A second XGBoost model is trained on the identical feature contract, but every peer-side quantity (deciles, facet peer-mean, tomatometer stays raw as an external feature) is built from the z-Split, and the target is $\hat{z}$; predictions are converted back via Equation 9 before scoring, using the same episode's μ_u, σ_u as the raw track (so raw and z rows are drawn from byte-identical sampled episodes).

5 Design 3: neural network

5.1 Raw

A residual MLP ensemble in PyTorch (Apple GPU/MPS), architecturally identical for both datasets: the 110/109 non-genre-embedding numeric columns (log1p-compressed where they are counts, NaN-imputed and standardized on train statistics) and a 24-dimensional genre embedding (`genre_id`) feed a projection into six pre-activation residual blocks of width 512 (≈3.25M weights; batch-norm,

GELU, dropout), then a linear head. Trained with AdamW, ReduceLROnPlateau, and early stopping; three independently seeded networks are averaged into an ensemble. Both trained models are verified leakage-free: peer means are leave-one-out, u/the visitor is excluded everywhere, and the facet-affinity term (Equation 7) is built only from seen films.

5.2 Z-score

A second ensemble, identical architecture, trained on the z-Split's rows with target $\hat{z}$; the three z-track members are seeded from a distinct offset so they are not literal duplicates of the raw-track members. Predictions convert back via Equation 9 before scoring, on the same paired test episodes as every other method.

6 Results

6.1 Rotten Tomatoes

Seen ratings n	3	5	10	20	50	all
B1 global mean	1.049	1.049	1.049	1.049	1.049	1.049
B2 Tomatometer → score	0.830	0.830	0.830	0.830	0.830	0.830
B3 reviewer mean	0.826	0.826	0.826	0.826	0.826	0.826
B4 top-10 similar mean	0.908	0.896	0.881	0.865	0.856	0.845
Design 1 (full)	0.964	0.922	0.864	0.833	0.811	**0.796**
Design 1 (top-|sim|)	0.989	0.948	0.889	0.858	0.828	0.808
Design 2 XGBoost	0.809	0.799	0.792	0.784	0.777	**0.770**
Design 3 neural net	0.813	0.801	0.793	0.785	0.778	**0.776**

Table 2: RMSE on the 43,344 paired, nested episodes (301 test critics, 2,894 pseudo-users total). Baselines are flat because they do not depend on the seen set.

Design 1 improves over B3 by 3.6% at full history; the trained models win everywhere: XGBoost overall/full-history RMSE 0.788/0.770, the neural net 0.791/0.776. The new top-|sim| variant of Design 1 trails the full-neighbourhood formula slightly at every seen-count (0.808 vs. 0.796 full-history) – restricting to 10 alignment-selected peers discards useful information the full weighted sum still captures, an honest negative result for that variant. Across low/mid/high critic-disagreement terciles, full-history gain over B2 is $+5.7/+6.2/+1.5\%$ for Design 1, $+7.8/+8.1/+6.2\%$ for XGBoost, $+7.2/+7.1/+5.6\%$ for the neural net.

Formula component (full-history episodes)	**RMSE**
Reviewer mean / movie consensus	0.850
Magnitude-scaled movie consensus	**0.833**
Movie-centered signed similarity ($g_{u,c} = 1$)	0.867
Full Design 1 formula	0.850

Table 3: Formula attribution: magnitude scaling supplies the whole gain over consensus.

Magnitude matching supplies the entire improvement over the consensus; the signed similarity term adds nothing on top – as with the trained models (§6.1), correlation-based taste matching contributes little.

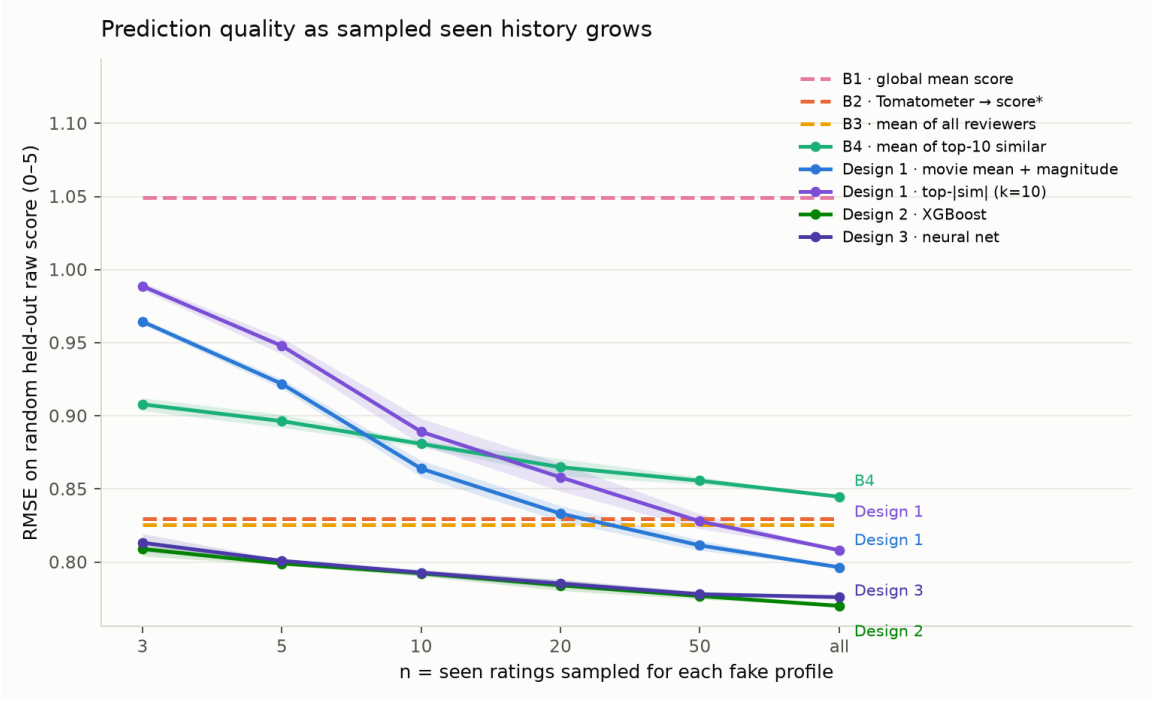

Figure 1: Error versus sampled seen history (Rotten Tomatoes). Both trained models beat every baseline at every seen-count.

Both trained models still lean first on the movie's Tomatometer/consensus, then the user's mean; scaling the network wider, deeper, and on more data moved the loss by nothing in earlier iterations of this experiment, and the newly added movie-facet features (§2.4) improved XGBoost's full-history RMSE only modestly (0.775→0.770) – on this engineered tabular feature set the ceiling is set by the information in the features, not model capacity, and facet-level personalization is a small effect on top of calibration, not a replacement for it.

6.2 Letterboxd

Seen ratings n	3	5	10	20	50	all
B1 global mean	2.108	2.108	2.108	2.108	2.108	2.108
B3 mean of all members	1.639	1.639	1.639	1.639	1.639	1.639
B4 top-10 similar mean	1.705	1.686	1.670	1.653	1.657	1.622
Design 1 (full)	1.765	1.670	1.597	1.561	1.545	**1.507**
Design 1 (top-|sim|)	1.836	1.738	1.649	1.592	1.573	1.526
Design 2 XGBoost	1.626	1.595	1.566	1.535	1.503	**1.455**
Design 3 neural net	1.642	1.610	1.581	1.538	1.515	**1.468**

Table 4: Letterboxd RMSE (1–10 scale) on the same paired, nested seen-history protocol: 300 deterministic test members, 8 fixed targets each, 3 seen-order redraws (41,166 episodes).

Design 1's top-|sim| variant trails the full formula here too (1.526 vs. 1.507 full-history) – the same honest result as Rotten Tomatoes, restricting the neighbourhood costs more than alignment-selection gains. XGBoost is now the best full-history method on Letterboxd (1.455, improved from

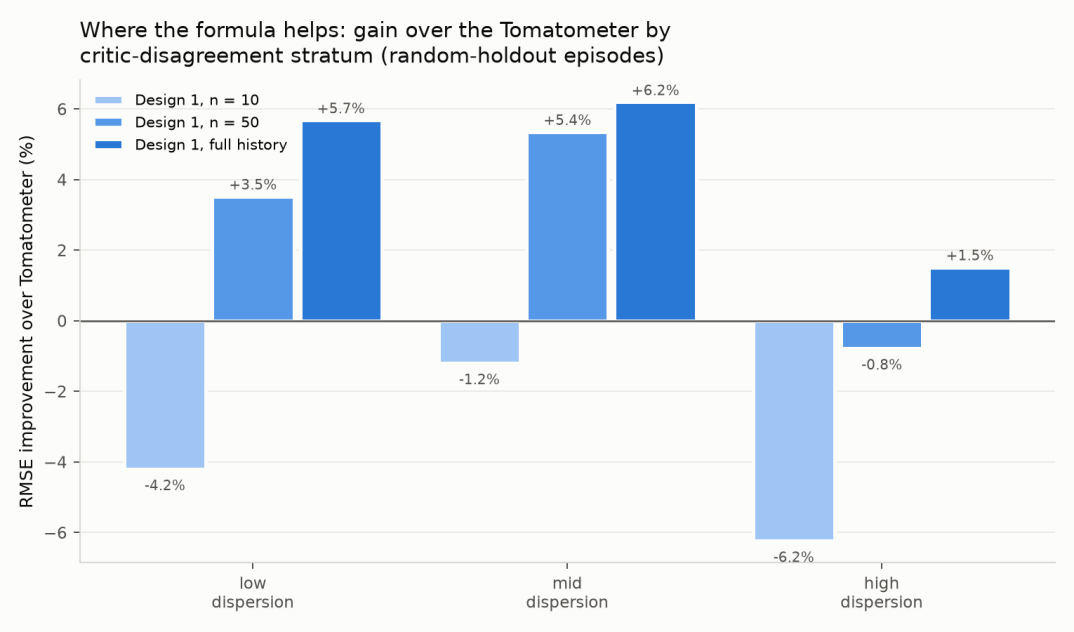

Figure 2: Design 1 gain over B2 by target-movie critic disagreement.

an earlier 1.502 pre-facet-feature baseline), narrowly ahead of the neural net (1.468).

Honest cross-dataset finding. Normalizing by rating range (RMSE / (max−min)) puts the two 0–5 and 1–10 scales on the same footing. Rotten Tomatoes' normalized RMSE is 0.159/0.162/0.154/0.155 (Design 1 full / Design 1 top-|sim| / XGBoost / neural net); Letterboxd's is 0.167/0.170/0.162/0.163. The two remain *comparable*, with Rotten Tomatoes still marginally *better* normalized despite training on far sparser critic pseudo-user profiles than Letterboxd's dense real member histories – more real rating data did not translate into a lower normalized error here, even with the richer movie-facet features added to both. Full numbers are in `results/letterboxd/cross_dataset_comparison.csv`.

6.3 Isolating scale: the z-score track

Design	raw RMSE	z RMSE (converted back)	z − raw
Rotten Tomatoes (0–5)			
Design 1 (full)	0.796	0.926	+0.130
Design 1 (top-|sim|)	0.808	0.817	+0.009
Design 2 XGBoost	0.770	0.778	+0.008
Design 3 neural net	0.776	0.781	+0.005
Letterboxd (1–10)			
Design 1 (full)	1.507	1.640	+0.134
Design 1 (top-|sim|)	1.526	1.528	+0.001
Design 2 XGBoost	1.455	1.455	+0.0004
Design 3 neural net	1.468	1.464	−0.004

Table 5: Full-history RMSE, raw vs. z-score track, both reported on the raw scale.

The result is **not uniform**, reported as found rather than presupposed. On Rotten Tomatoes the z-track trails raw at every design; the full-neighbourhood Design 1 loses the most (+0.130) since it

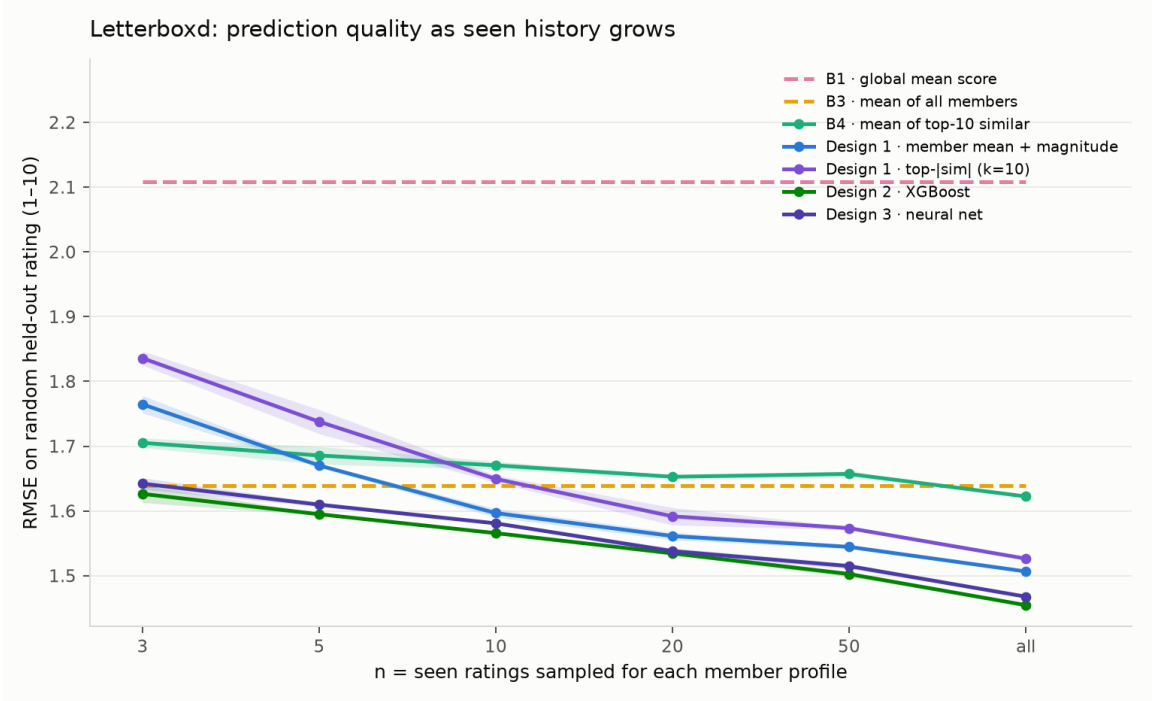

Figure 3: Letterboxd: error versus sampled seen history, styled identically to Figure 1.

has no learned capacity to reallocate, while the top-|sim| variant and both trained models lose far less (+0.005 to +0.009). On Letterboxd, the trained models (Design 2/3) come out *flat-to-slightly-better* in z-space, and even the top-|sim| analytic variant is nearly indifferent (+0.001) – only the full-neighbourhood formula still loses meaningfully. A plausible reading: restricting to the top-|sim| peers, or handing a model spare learned capacity, both let the deviation-only signal be used more directly once the rater's own level is removed; the full-neighbourhood analytic formula never had that capacity to reallocate, so it only loses the calibration information without regaining anything. Either way, the effect is small relative to raw calibration's gain over the flat baselines (Table 2, 4) – calibration, not taste-matching, remains the dominant effect in this data.

7 Similar-film suggestions: does rating them help?

Each row of the app's "films to predict" table offers a "movies like this one" dropdown: rate one of the target film's nearest content neighbours and the prediction should, in principle, improve. Neighbours are precomputed offline with K-means: a standardized feature vector per catalog film (the genre/decade multi-hot from §2.4, a catalog-scoped top-k multi-hot over studio/director/actor/theme/language/country, and numeric runtime/external-rating/year/consensus), clustered into $\max(2, \lfloor n/25 \rfloor)$ groups, with each film's `k_neighbors` nearest neighbours (Euclidean distance) taken from within its own cluster (topped up from the globally nearest remaining films if the cluster is too small). `k_neighbors` is the dropdown's list length, identical on both datasets.

Does it work? 6,000 paired (rater, n=10 seen, target) episodes were drawn from each project's own 1,000-film catalog (Design 1, leave-one-out, identical protocol to §3) and bucketed by how many of the target's `k_neighbors` nearest neighbours were already in that episode's seen set – the same 6,000 predictions are reused for every `k_neighbors` value, so only the bucketing (which films

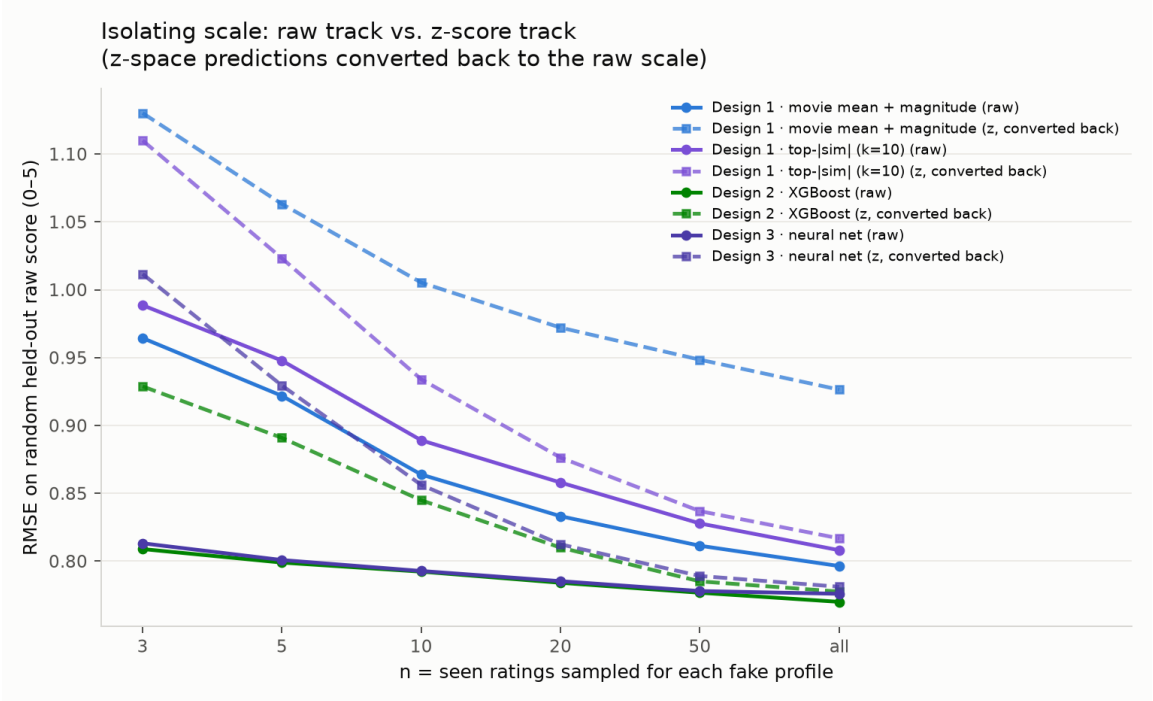

Figure 4: Rotten Tomatoes: raw (solid) vs. z-score (dashed) RMSE by seen count, per design.

count as "similar") differs.

On **Rotten Tomatoes** the signal is present but noisy. At k=20 the per-bucket RMSE by similar-seen count is 1.136 (0 seen, n=4,435), 1.139 (1), 0.987 (2), 1.220 (3), 0.795 (4, n=10) – a downward *tendency* as more of the target's similar films are rated, but non-monotonic, and the higher-count buckets are thinly populated. Pooling every episode with at least three similar films seen and dropping buckets below five episodes gives the cleanest single summary: the episode-count-weighted average of those buckets' RMSE is 1.121 at k=20 (n=43), below k=30's 1.525 (n=99) and k=10's 3.966 (n=7, too thin to trust). On this modest evidence k=20 is the best-supported choice for Rotten Tomatoes.

On **Letterboxd**, the same sweep is noisier: RMSE does not fall monotonically with similar-seen count for any `k_neighbors`, and the mechanical "lowest RMSE at $\geq$3 similar seen" rule picks k=10 (0.622) – but that bucket has only n=6 episodes, an order of magnitude thinner than Rotten Tomatoes' equivalent and not a result to trust. k=20's own bucket (2.698, n=18) is more populated but still noisy, and k=30 (2.986, n=45) does not improve on it. **Reported honestly**: Letterboxd's own sweep does not cleanly discriminate among `k_neighbors` values at this sample size, unlike Rotten Tomatoes' cleaner (if still modest) signal. `k_neighbors`= 20 was set for *both* projects – Rotten Tomatoes' larger, more reliable sample supports it outright, and Letterboxd's own data does not contradict it strongly enough to justify a different value per project. Full per-bucket numbers are in `results/{rotten_tomatoes,letterboxd}/similar_k_sweep.csv` (`rotten_tomatoes.design1_analytic.similar_k_sweep` / `letterboxd.similar_k_sweep`).

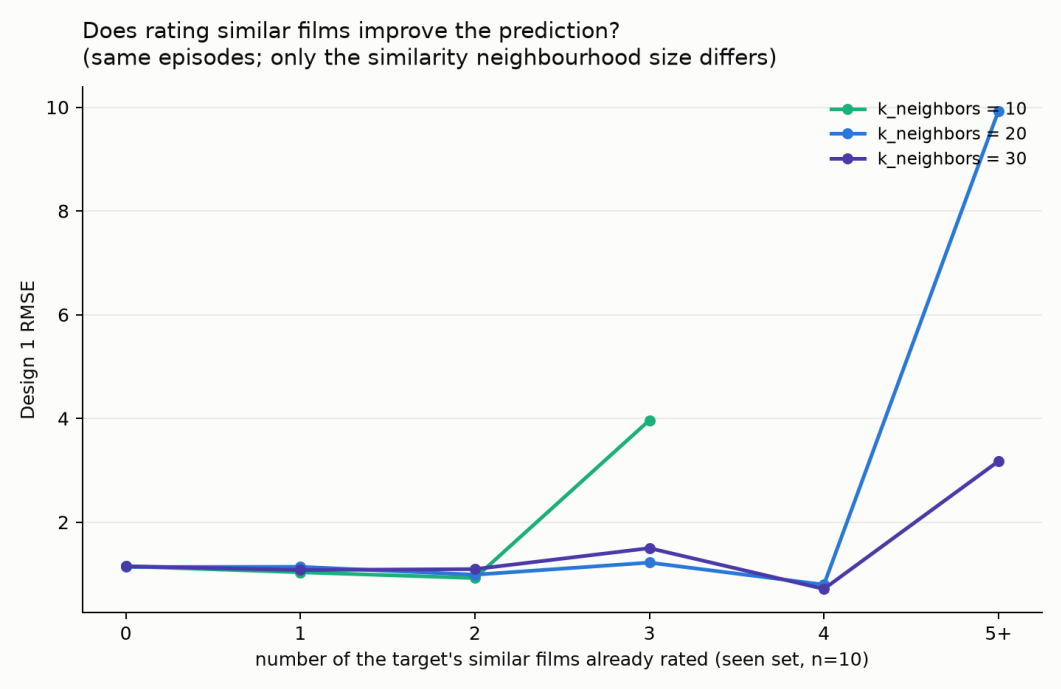

Figure 5: Rotten Tomatoes: Design 1 RMSE by how many of a target's `k_neighbors` nearest neighbours are already rated, for `k_neighbors` $\in \{10, 20, 30\}$.

8 Application and limitations

The website (Table 1: 1,000 catalog movies, 3,387 critics / 7,416 members) is organized in three sections, with a sort control (alphabetical, year, most-reviewed) and a **genre filter** (a dropdown built from each film's gsimonx37 genre list, §1.1–1.2) over the search lists. *(1) Films you have seen*: a search box adds a film to a table where each row carries a rating widget (five-star on Rotten Tomatoes, a 1–10 stepper on Letterboxd). *(2) Films to predict*: the same searchable table, scoring optional; each row also has a "movies like this one" dropdown (§7) – green if you have already rated one of its 20 nearest content neighbours, red otherwise. *(3) Predictions*: for every target film the app shows all four model variants (Design 1 full, Design 1 top-|sim|, XGBoost, neural net) side by side, each with its raw and z-score prediction, plus the movie's consensus mean and (Rotten Tomatoes only) Tomatometer. When the visitor scores some target films, the app reports the mean squared error of every variant against the visitor's own score and marks the closest, answering "which method predicts me best?". All eight model variants (four designs × two tracks) run entirely client-side, ported from the Python training code to hand-written TypeScript (an XGBoost tree-walker, a neural-net forward pass, and the movie-facet affinity computation, verified to match Python to within numerical tie-break tolerance – see `web/scripts/validate*.ts`).

Two secondary metrics (`comparison.analysis`, `secondary_metrics.csv`) check that the gain is calibration rather than reordering: within-user Spearman rank correlation between predictions and held-out ratings, and precision@10. Design 1's within-user Spearman is 0.578 versus 0.580 for the reviewer mean (B3) – ranking is barely changed, consistent with the attribution finding (Table 3) that the win is calibration, not reordering a user's films.

Item holdout is not temporal forecasting. The gsimonx37 join is fuzzy (normalized title + year) and RT's own release-date coverage is sparse, so its match rate (69.2% full catalog) is lower than

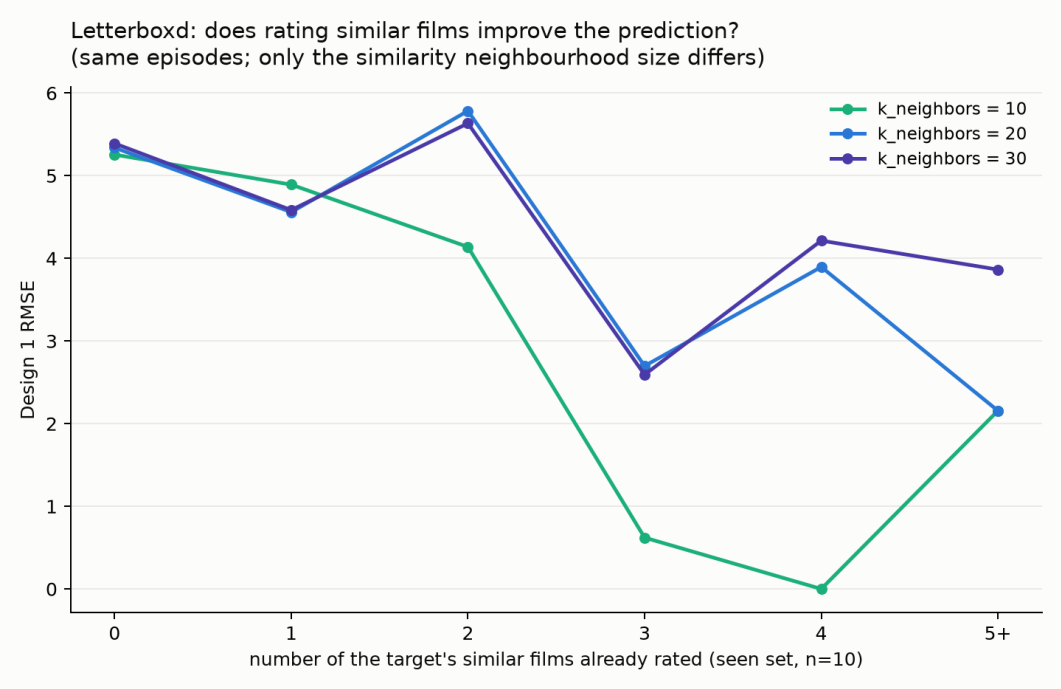

Figure 6: Letterboxd: the same sweep, styled identically to Figure 5.

Letterboxd's (94.3%) – reported honestly rather than assumed complete; unmatched films fall back to each project's own (sparser) genre parsing. The top-|sim| analytic variant underperforms the full formula on both datasets – a negative result, not tuned away. The magnitude multiplier is an unregularized through-origin slope. Scores are quantized to six levels on Rotten Tomatoes; pseudo-users there are professional critics rather than casual users.

Repository layout

Each Rotten Tomatoes design lives in its own package under `src/rotten_tomatoes/`: `design1_analytic/` (`analytic.py` the formula, top-k baseline, and the new top-|sim| variant; `run.py` the k-sweep and paired test; `attribution.py` the formula-component decomposition; `similar_k_sweep.py` the neighbour-list tuning of §7; `predict.py` app inference), `design2_xgboost/train.py`, `design3_neural/` (`network.py`, `train.py`), and `comparison/analysis.py`. The pseudo-user substrate (`pseudo_users.py`), the shared feature contract (`features.py`), and the gsimonx37 movie-facet join (`movie_features.py`) live once at the `rotten_tomatoes` package root and are imported by all three designs. `src/letterboxd/` mirrors this layout exactly (its own `features.py`, `movie_features.py`, `train_xgboost.py`, `train_neural.py`, `analyze.py`, `web_export.py`), sharing no code, data, or model artifacts with Rotten Tomatoes – the gsimonx37 CSVs are the one shared external download, read independently by each project's own join. `tests/` covers the formula, the random-holdout and paired-episode protocols, partition disjointness, and the feature contract including the facet-affinity leave-target-out property.

Reproducibility. Python, pandas, scipy.sparse, NumPy, XGBoost, PyTorch, and matplotlib. The pseudo-user substrate and feature code are shared, seeded modules (see Repository layout above), so the cross-design comparison is exact. Both projects run entirely client-side in the browser (Vite/React/TypeScript, `web/`), hosted as a static GitHub Pages site with no server. Run the ordered module pipelines listed in the README

from `src/`; all random seeds are fixed.